

От кода к архитектуре: Комплексный теоретический курс C++ для коллоквиума

Событийно-ориентированное программирование и графика с SFML

Событийно-ориентированное программирование представляет собой парадигму, в которой поток выполнения программы определяется и реагирует на события [11](#). Эти события могут генерироваться пользовательским вводом, например, нажатием клавиш на клавиатуре или движением мыши, либо системными действиями, такими как закрытие окна приложения или изменение его размера. Эта модель является стандартом де-факто для создания интерактивных приложений, включая игры, графические редакторы и современные десктопные программы. Ключевой особенностью EDP является отсутствие единого последовательного пути выполнения кода; вместо этого программа ждет наступления событий и выполняет соответствующий блок кода (обработчик события) каждый раз, когда событие происходит.

Центральным элементом любого приложения, основанного на этой парадигме, является основной цикл программы, часто называемый игровым циклом. Этот цикл представляет собой бесконечный цикл, который продолжает свою работу до тех пор, пока окно приложения остается открытым. Его работа обычно организуется с помощью конструкции `while(window.isOpen()) { ... }`. Внутри этого цикла выполняются три фундаментальных этапа, которые повторяются с высокой частотой, формируя один кадр: обработка событий, обновление логики и состояние игры (или приложения), и наконец, отрисовка кадра на экран. Этот цикл является сердцем реактивности приложения, постоянно анализируя входящие сигналы и корректируя его внутреннее состояние и визуальное представление в ответ на них.

Для обеспечения плавной работы и предотвращения чрезмерного потребления процессорного времени необходимо контролировать частоту кадров. Избыточная скорость обновления приводит к тому, что CPU работает на

максимальной мощности, не имея ничего полезного для выполнения, что неэффективно и может вызвать перегрев. Библиотека SFML предоставляет удобный инструмент для решения этой задачи — метод `setFramerateLimit(60)`. Этот метод устанавливает верхнюю границу частоты кадров в 60 кадров в секунду. Под капотом SFML использует класс `sf::Clock` для точного измерения времени, прошедшего с начала текущего кадра, и, если расчетное время для одного кадра ($1/60$ секунды) еще не истекло, вызывает функцию `sf::sleep` для приостановки выполнения программы на оставшееся время. Это позволяет циклу выполнять свою работу с регулярным интервалом, обеспечивая стабильную производительность и комфортное восприятие для пользователя. Для более тонкого контроля над временем можно использовать класс `sf::Time`, который позволяет самостоятельно вычислять длительность кадра и реализовывать собственную логику управления скоростью обновления.

Еще одной важнейшей техникой, обеспечивающей качество визуального вывода в графических приложениях, является двойная буферизация. Мерцание экрана, которое возникает при поочередном рисовании каждого элемента прямо на видимый экран, является неприятным зрительским эффектом. Двойная буферизация решает эту проблему, используя два буфера памяти: один видимый буфер, который отображается на экране, и один скрытый буфер, в который происходит вся работа по рендерингу. Все примитивы, фигуры и текстуры рисуются не на экран, а во вторичный буфер. После того как вся сцена для данного кадра была успешно отрисована, происходит мгновенная замена содержимого этих двух буферов. Скрытый буфер становится видимым, а старый видимый буфер очищается и готовится к следующему кадру. Этот быстрый обмен местами делает весь отрисованный кадр одновременно видимым для пользователя, создавая иллюзию плавного и непрерывного движения. Хотя конкретная реализация в SFML не детализируется в предоставленных источниках, это общепринятый подход в большинстве современных графических API, и SFML следует этому стандарту по умолчанию.

Для повышения качества изображения, особенно при отрисовке геометрических примитивов, таких как линии и многоугольники, используется антиалиасинг. Этот процесс направлен на сглаживание "ступенчатых" краев (эффекта "зубчатости"), который возникает из-за дискретной природы пикселей на экране. Антиалиасинг работает, смешивая цвет пикселя на границе объекта со смежными цветами, чтобы создать переходную область, которая менее заметна человеческому глазу. В SFML антиалиасинг активируется на уровне контекста окна. При создании объекта `sf::RenderWindow` можно передать третий параметр — объект `sf::ContextSettings`. В этом объекте можно установить

количество сэмплов для сглаживания. Например, `sf::ContextSettings settings; settings.antiAliasingLevel = 4; sf::RenderWindow window(..., settings);` включит четырехкратный антиалиасинг. Этот процесс требует дополнительных вычислений и увеличивает нагрузку на видеокарту, поэтому существует компромисс между визуальным качеством и производительностью.

В любой графической среде критически важно понимать системы координат, поскольку именно они определяют положение всех объектов на экране. SFML использует несколько систем координат. Во-первых, это пиксельная система координат, где положение точки задается целочисленными координатами (x, y) относительно верхнего левого угла окна. Во-вторых, существует глобальная система координат, в которой находятся все объекты в окне, и каждому объекту можно задать свои локальные координаты. Локальные координаты относятся к самому объекту, в то время как глобальные — к окну. Это становится особенно важным, когда объекты перемещаются, поворачиваются или масштабируются. Для преобразования координат между этими системами SFML предоставляет специальные методы:

`window.mapPixelToCoords(pixelPosition)` преобразует пиксельные координаты в координаты в системе мира (глобальные), а `window.mapCoordsToPixel(worldPosition)` делает обратное преобразование. Эти функции жизненно необходимы для корректной обработки взаимодействия с объектами, например, для определения, на какой именно спрайт нажал пользователь мышью, даже если этот спрайт находится в движении.

В контексте обработки пользовательского ввода в SFML существует два основных подхода: опрос состояний и обработка событий. Первый подход, использующий функции вроде

`sf::Keyboard::isKeyPressed(sf::Keyboard::Key::W)` и `sf::Mouse::isButtonPressed(sf::Mouse::Left)`, прост и нагляден. Он отвечает на вопрос "Нажата ли эта клавиша/кнопка прямо сейчас?". Это аналог состояния кнопки на пульте дистанционного управления: она либо нажата, либо нет. Такой подход хорошо подходит для определения постоянного состояния, например, для реализации непрерывного движения персонажа, пока игрок удерживает клавишу W. Однако он не предоставляет информации о самом моменте нажатия или отпускания.

Второй подход, основанный на обработке событий, гораздо более информативен. SFML помещает все события (заккрытие окна, нажатие клавиши,

движение мыши и т.д.) в очередь событий. В начале каждого цикла программа должна опрашивать эту очередь в цикле `while (window.pollEvent(event)) { ... }`. Внутри цикла по объекту `event` можно проверить его тип (`event.type`) и, в зависимости от типа, извлечь дополнительную информацию, например, код клавиши (`event.key.code`) или координаты мыши (`event.mouseMove.x`, `event.mouseMove.y`). Этот подход реагирует не на состояние, а на факт события. Это позволяет точно определить момент начала и конца взаимодействия. Например, для запуска стрельбы достаточно проверить событие `MouseButtonPressed`, а для запуска движения — событие `KeyPressed`. Этот механизм также поддерживает автоповтор клавиш, генерируя повторяющиеся события `KeyPressed` с определенной скоростью после первого нажатия, что задается параметрами `Repeat delay` и `Repeat rate`.

На основе этих концепций можно реализовать простейшие элементы графического интерфейса. Например, класс "перетаскиваемой таблички" будет хранить свойство, указывающее, захватила ли мышь его захвата. Обработчик события `MouseButtonPressed` будет проверять, находится ли курсор мыши внутри границ таблички. Если да, то устанавливается флаг захвата. Затем, в цикле обработки событий, обработчик `MouseMoved` будет проверять этот флаг. Если он установлен, координаты таблички будут обновляться на основе разницы между новой и старой позицией мыши. Флаг снимается при событии `MouseButtonReleased`. Аналогично, класс "кнопки" будет проверять в обработчике `MouseButtonPressed` попадает ли курсор в ее область. Если да, выполняется действие кнопки, а затем, чтобы избежать множественного срабатывания, проверяется `MouseReleased` для завершения цикла взаимодействия. Реализация "слайдера" усложняется, но принцип остается тем же: обработка захвата за "ручку" слайдера и обновление его значения на основе движения мыши в рамках ограничений слайдера.

Метод / Концепция	SFML-реализация	Назначение
Основной цикл	<code>while(window.isOpen()) { ... }</code>	Сердце EDP-приложения, управляющий обработкой событий, логикой и отрисовкой.
Ограничение FPS	<code>window.setFramerateLimit(60)</code>	Установка максимального количества кадров в секунду для стабильной производительности.
Управление временем	<code>sf::Clock</code> , <code>sf::Time</code> , <code>sf::sleep</code>	Точное измерение времени для реализации собственной логики управления кадрами.
Двойная буферизация	Встроено в SFML по умолчанию	Предотвращение мерцания экрана путем отрисовки в скрытом буфере и его быстрой смены с видимым.
Антиалиасинг	<code>sf::ContextSettings.antialiasingLevel</code>	Сглаживание "ступенек" на краях фигур для повышения качества изображения.
Системы координат	Пиксельная, глобальная, локальные	Определение положения объектов на экране и в мире.
Конвертация координат	<code>window.mapPixelToCoords()</code> , <code>window.mapCoordsToPixel()</code>	Преобразование координат между пиксельной системой и системой мира.
Опрос состояний	<code>sf::Keyboard::isKeyPressed()</code> , <code>sf::Mouse::isButtonPressed()</code>	Проверка нажатия клавиши/кнопки в данный момент времени.
Обработка событий	<code>window.pollEvent(event)</code> , цикл <code>while</code>	Реакция на факты нажатия/отпускания клавиш и движения мыши.

Механизмы наследования и композиции в C++

Наследование в языке C++ является одним из ключевых механизмов объектно-ориентированного программирования, позволяющим создавать новый класс на основе уже существующего. Этот механизм способствует переиспользованию кода и созданию иерархических структур классов. Класс, от которого производится наследование, называется базовым (или родительским), а новый класс — производным (или дочерним). Производный класс автоматически получает доступ к членам базового класса в соответствии с модификаторами доступа, а также может расширять его функциональность, добавляя новые поля данных и методы или переопределяя существующие. Синтаксически наследование в C++ объявляется с использованием двоеточия после имени класса, за которым следует доступитель и имя базового класса, например: `class Derived : public Base`. Здесь `public` — это один из трех возможных модификаторов доступа при наследовании [3](#) [76](#).

Одной из наиболее тонких и часто ошибочно понимаемых концепций наследования является затенение. Затенение происходит, когда производный класс определяет член (поле или метод) с тем же именем, что и член базового класса. В этом случае компилятор "скрывает" (или "затеняет") член базового класса для объектов производного класса. Важно различать затенение и перегрузку. Перегрузка — это определение нескольких функций с одинаковым именем, но разными списками параметров в одном и том же `scope`. Затенение же происходит, когда сигнатуры функций совпадают, но один находится в базовом, а другой — в производном классе. Если мы вызовем метод с этим именем через указатель или ссылку на производный класс, будет вызван именно метод производного класса, скрывший базовый. Чтобы получить доступ к скрытому члену базового класса, необходимо явно указать его принадлежность, например, `Base::methodName()`. Для того чтобы избежать необходимости вручную перечислять все имена членов базового класса, которые должны быть доступны в производном, используется ключевое слово `using`. Инструкция `using Base::methodName;` в теле производного класса "подтягивает" указанное имя в его область видимости, отменяя затенение для этого конкретного члена [29](#).

Процесс создания и уничтожения объектов, использующих наследование, строго регламентирован и имеет решающее значение для корректной работы программы. При создании объекта производного класса сначала всегда вызываются конструкторы всех базовых классов (в том порядке, в котором они объявлены в списке наследования), а только потом — конструктор производного класса [34](#) [36](#) [62](#). Это гарантирует, что объект-часть (базовый класс) полностью инициализирован прежде, чем начнется инициализация объекта-целого (производного класса). Аналогично, при уничтожении объекта порядок вызова деструкторов обратен порядку вызова конструкторов: сначала вызывается деструктор производного класса, а затем — деструкторы всех его базовых классов [2](#) [36](#). Это необходимо, чтобы в деструкторе производного класса можно было безопасно обращаться к членам базового класса, которые еще не были уничтожены. Если конструктор производного класса не указывает, какой конструктор базового класса следует вызвать, будет использован конструктор по умолчанию. Если базовый класс не имеет конструктора по умолчанию, компилятор выдаст ошибку. Чтобы явно указать, какой конструктор базового класса нужно вызвать, используется список инициализации в конструкторе производного класса: `Derived::Derived() : Base(arg1, arg2) { ... }`. Также, начиная с C++11, ключевое слово `using` можно применять к конструкторам, чтобы "наследовать" все перегрузки конструкторов

базового класса в производный, что значительно упрощает код: `using Base::Base;`

Модификаторы доступа при наследовании (`public`, `protected`, `private`) определяют, как доступ к членам базового класса изменяется в производном. Это мощный инструмент для построения чистых и понятных интерфейсов API.

- Публичное наследование (`public`): `public` и `protected` члены базового класса остаются `public` и `protected` в производном. `private` члены базового класса остаются недоступными. Это наиболее распространенный вид наследования, который моделирует отношение "является" (A is a B).
- Защищенное наследование (`protected`): `public` и `protected` члены базового класса становятся `protected` в производном. Это означает, что они будут доступны для дальнейших производных классов, но недоступны для кода за пределами иерархии.
- Приватное наследование (`private`): `public` и `protected` члены базового класса становятся `private` в производном. Это самый строгий уровень доступа. Часто приватное наследование рассматривается как избыточное и его функциональность может быть достигнута более элегантно с помощью композиции [52](#) [53](#).

Важно отметить, что друзья базового класса никогда не получают доступа к приватным полям производного класса, так как `friend` — это отношение, не обладающее транзитивностью [53](#).

Одной из главных проблем, связанных с наследованием, особенно с множественным, является проблема ромбовидного наследования (Diamond Problem). Она возникает, когда класс `D` наследуется от двух классов `B` и `C`, а классы `B` и `C` обоих наследуются от общего базового класса `A`. В результате производный класс `D` получает две отдельные копии (экземпляра) класса `A`: одну через `B` и другую через `C`. Это приводит к неоднозначности: если попытаться обратиться к члену класса `A` через объект `D`, компилятор не сможет определить, к какой из двух копий обращаться. Кроме того, это приводит к увеличению занимаемой памяти. Язык C++ решает эту проблему с помощью виртуального наследования. Объявив наследование от `A` в классах `B` и `C` как виртуальное (`class B : virtual public A`), мы говорим компилятору, что в любой иерархии наследования должен существовать лишь один общий экземпляр класса `A`. Классы `B` и `C` больше не содержат свою копию `A`; вместо этого они хранят информацию для управления единственным экземпляром, который будет создан и инициализирован непосредственно при создании

объекта самого производного класса D. Это эффективно устраняет неоднозначность и проблемы с памятью, хотя и усложняет внутреннее представление объекта в памяти, добавляя дополнительные указатели для вычисления правильных смещений адресов [4](#) [5](#) [58](#) [74](#).

Помимо наследования, существует второй фундаментальный механизм композиции классов в C++, который часто рекомендуется к использованию. Принцип "Предпочтите композицию наследованию" стал практически каноническим советом в ООП [30](#) [59](#). Его суть заключается в том, что вместо того, чтобы заимствовать и расширять поведение через наследование, класс должен содержать в себе объект другого класса в качестве своего члена. Это позволяет достигать схожих результатов с большей гибкостью и меньшим уровнем связанности. Преимущества композиции включают:

- **Гибкость:** Поведение можно изменять во время выполнения, заменяя объект-член на другой. С наследованием это возможно только на этапе компиляции (через полиморфизм).
- **Меньшая связанность:** Класс, использующий композицию, зависит от интерфейса содержащегося объекта, но не от его конкретной реализации. Это снижает зависимость от внутренней структуры и деталей другого класса.
- **Избегание "хрупкого базового класса":** Любые изменения в базовом классе могут случайно сломать все его производные классы. При композиции такой проблемы нет, так как зависимость идет только от публичного интерфейса, а не от реализации.
- **Четкость кода:** Композиция часто делает отношения между классами более явными и легкими для понимания. В то время как наследование моделирует отношение "является", композиция моделирует отношение "состоит из" [31](#).

Аспект	Наследование	Композиция
Отношение	"Является" (Is-a)	"Состоит из" (Has-a)
Уровень связанности	Высокий	Низкий
Гибкость	Низкая (изменения на этапе компиляции)	Высокая (изменения на этапе выполнения)
Реализация	<code>class Derived : public Base</code>	<code>class Container { Base member; }</code>
Зависимость от базового класса	Высокая ("хрупкий базовый класс")	Низкая (зависит от интерфейса)
Основной принцип	Расширяемость и полиморфизм	Гибкость и слабая связанность 30 32

Динамический полиморфизм и информация о типе во время выполнения (RTTI)

Динамический полиморфизм — это одна из столпов объектно-ориентированного программирования, предоставляющая возможность объектам разных классов реагировать на один и тот же вызов функции по-разному [11](#). В отличие от статического полиморфизма, который реализуется шаблонами и перегрузкой функций на этапе компиляции, динамический полиморфизм определяет, какой именно метод будет вызван, уже во время выполнения программы. Это достигается за счет использования виртуальных функций и указателей (или ссылок) на базовые классы. Ключевое преимущество динамического полиморфизма заключается в его способности создавать гибкие и легко расширяемые системы, где можно добавлять новые типы объектов, не изменяя существующий код, который с ними работает.

Основой механизма динамического полиморфизма является концепция виртуальной функции. Если в классе объявляется хотя бы одна функция с ключевым словом `virtual`, объекты этого класса (и всех его производных) становятся полиморфными. Для каждого такого класса компилятор создает специальную структуру данных — таблицу виртуальных функций (`vtable`). Это массив указателей на функции. В эту таблицу заносятся адреса реализации всех виртуальных функций класса. Каждый объект полиморфного класса получает в свое составе скрытое поле-указатель (`vptr`), который при создании объекта инициализируется адресом соответствующей `vtable`. Когда через указатель на базовый класс вызывается виртуальная функция, программа не выполняет вызов напрямую. Вместо этого она по `vptr` извлекает адрес нужной функции из `vtable` и выполняет его. Таким образом, если указатель на базовый класс `Base*` указывает на объект производного класса `Derived`, а виртуальная функция переопределена в `Derived`, то в `vtable` объекта `Derived` будет адрес функции из `Derived`, и именно она будет вызвана [38](#) [78](#). Размер объектов полиморфных классов всегда больше размера обычных объектов из-за наличия этого скрытого указателя `vptr`. Важно понимать, что при работе программы в памяти существует только одна копия каждой `vtable` для данного класса, а не по одной для каждого объекта [38](#).

Виртуальные функции играют центральную роль в жизни объектов, особенно в контексте конструкторов и деструкторов. Вызов виртуальной функции из конструктора или деструктора базового класса является синтаксически

допустимым, но поведение в этом случае неинтуитивно: в момент вызова из конструктора производного класса, объект еще не является полноценным объектом производного класса, и вызов виртуальной функции будет направлен к ее реализации в самом базовом классе, а не в производном. Однако есть одно исключение: деструкторы являются виртуальными по умолчанию в иерархии полиморфных классов. Если деструктор базового класса не является виртуальным, а на него указывает указатель, а на самом деле на объект производного класса, то вызов `delete` на этом указателе приведет к неопределенному поведению. Программа вызовет деструктор базового класса, но не вызовет деструктор производного класса, что приведет к утечке памяти и ресурсов, выделенных в производном классе [17](#) [19](#). Поэтому, если класс содержит хотя бы одна виртуальная функция, его деструктор должен быть объявлен как виртуальный. Можно также объявить деструктор как "чисто виртуальный" (`virtual ~Base() = 0;`), что делает класс абстрактным и обязывает производные классы предоставлять свою реализацию деструктора [17](#) [19](#).

Когда все виртуальные функции в классе объявлены как "чисто виртуальные" (например, `virtual void func() = 0;`), такой класс называется абстрактным классом. Он не может быть использован для создания экземпляров. Его основное предназначение — служить интерфейсом или шаблоном для производных классов. Любой производный класс, который хочет быть конкретным (неабстрактным), обязан переопределить все чисто виртуальные функции базового. Если он этого не делает, то сам становится абстрактным классом. Интерфейс в C++ — это класс, содержащий только публичные чисто виртуальные функции. Множественное наследование от нескольких таких интерфейсов позволяет реализовать поведение, схожее с множественным наследованием от интерфейсов, как в Java или C#. Попытка вызвать чисто виртуальную функцию из абстрактного класса, которая не была переопределена в конкретном производном классе, приведет к ошибке времени выполнения, известной как "pure virtual function call".

Для реализации многих полиморфных операций, особенно для безопасного приведения типов, необходим механизм, позволяющий узнать тип объекта во время выполнения. Этот механизм называется информацией о типе во время выполнения (RTTI). RTTI предоставляет два основных инструмента: оператор `typeid` и оператор `dynamic_cast`. Для того чтобы RTTI работал корректно, класс должен быть полиморфным, то есть иметь хотя бы один виртуальный метод. Без виртуальных функций `typeid` может работать только со

статическими типами [21](#) [24](#) [39](#). Оператор `typeid` принимает в качестве операнда переменную или тип и возвращает константную ссылку на объект класса `std::type_info`, который содержит информацию о типе. Этот объект можно использовать для сравнения типов [23](#) [67](#).

Однако самым мощным инструментом RTTI является оператор `dynamic_cast`. Он предназначен для безопасного приведения указателей и ссылок в иерархии наследования. В отличие от `static_cast`, который выполняет проверку типов на этапе компиляции и может быть опасен, `dynamic_cast` выполняет проверку во время выполнения [37](#). Он работает только с указателями на полиморфные типы. Если `dynamic_cast` используется для приведения указателя, и преобразование невозможно, он просто возвращает `nullptr`. Если он используется для приведения ссылки, и преобразование невозможно, он выбрасывает исключение `std::bad_cast` [22](#) [73](#). Это делает его идеальным инструментом для безопасного "спуска" по иерархии наследования. Например, если у нас есть `Base<em> ptr`, и мы хотим использовать метод `DerivedSpecificClass`, только если `ptr` действительно указывает на объект этого класса, мы можем написать: `if (DerivedSpecificClass</em> derivedPtr = dynamic_cast<DerivedSpecificClass*>(ptr)) { ... }`. Использование `static_cast` для такого преобразования может привести к неопределенному поведению, если указатель на самом деле не указывает на корректно преобразуемый объект [1](#) [56](#).

Характеристика	<code>static_cast</code>	<code>dynamic_cast</code>	<code>const_cast</code>	<code>reinterpret_cast</code>
Назначение	Обратимые преобразования типов, преобразования в иерархии наследования (вверх)	Безопасное преобразование в иерархии наследования (вниз и по диагонали)	Удаление <code>const</code> -кастности	Любые преобразования указателей
Проверка	На этапе компиляции	Во время выполнения (требуется RTTI)	На этапе компиляции	На этапе компиляции
Безопасность	"Слепое", может быть неопределенным поведением	"Осмотрительное", безопасное для указателей (возвращает <code>nullptr</code>)	Безопасно для удаления <code>const</code>	Очень опасно, небезопасно
Требования	Соответствие правилам преобразования типов	Полиморфный тип (хотя бы один виртуальный метод)	Исходный тип должен быть <code>const-qualified</code>	Любой тип указателя или целочисленного

Основы проектирования: Принципы и архитектурные решения

Создание больших и сложных программных систем требует не только знания синтаксиса языка, но и глубокого понимания архитектурных принципов, которые помогают создавать код, легкий в поддержке, тестировании и расширении. Эти принципы представляют собой набор здравых правил, выработанных сообществом опытных разработчиков, и направлены на управление сложностью. Одним из самых влиятельных наборов таких принципов является мнемоника **SOLID**, предложенная Робертом Мартином. Каждая буква в SOLID представляет один из пяти фундаментальных принципов объектно-ориентированного проектирования.

S — Принцип единственной ответственности (**Single Responsibility Principle, SRP**). Этот принцип гласит, что каждый модуль или класс должен иметь только одну причину для изменения. Другими словами, класс должен решать только одну задачу. Если у класса есть более одной причины для изменения, его ответственность следует разделить между несколькими классами. Например, класс, который одновременно хранит данные о пользователе и отправляет уведомления по электронной почте, имеет две ответственности: хранение данных и коммуникация. Если изменится политика отправки писем, придется менять класс, даже если данные пользователя не менялись. Разделив эти две обязанности на два класса, мы сделаем код более устойчивым к изменениям. Этот принцип тесно связан с идеей "Инкапсулируйте то, что может измениться" [48 49](#).

O — Принцип открытости/закрытости (**Open/Closed Principle, OCP**).

Программные сущности (классы, модули, функции) должны быть открыты для расширения, но закрыты для модификации. Это означает, что код должен быть спроектирован таким образом, чтобы добавлять новую функциональность можно было, не трогая существующий код. Жесткая привязка к конкретным реализациям делает систему хрупкой и трудной для поддержки. Основным инструментом для достижения этого принципа является динамический полиморфизм. Например, вместо того чтобы писать длинный условный блок `if-elif` для выбора алгоритма, можно определить общий интерфейс "Стратегия" и создавать объекты различных классов-стратегий, которые реализуют этот интерфейс. Клиентский код работает только с интерфейсом, и чтобы добавить новый алгоритм, достаточно создать новый класс-наследник, не изменяя ни одной строки существующего кода [10](#).

L — Принцип подстановки Барбары Лисков (Liskov Substitution Principle, LSP).

Этот принцип гласит: "Если в программе вместо объекта суперкласса можно использовать объект подкласса, то программа будет работать корректно".

Иными словами, производные классы должны быть взаимозаменяемы с базовыми классами. Это означает, что производный класс не должен нарушать контракт, установленный базовым классом. Он может расширять функциональность, но не может ее ограничивать или изменять ее поведение таким образом, что это нарушит логику, ожидающую объект базового класса. Например, если у нас есть класс `Rectangle` с методами `setWidth` и `setHeight`, и мы создаем от него класс `Square`, который наследует эти методы, но при установке ширины он также изменяет высоту, мы нарушим LSP. Код, который рассчитывает площадь прямоугольника по формуле `width * height`, сломается, если ему передать объект `Square`, потому что поведение `setWidth` изменилось. Нарушение этого принципа приводит к "хрупкому базовому классу", когда любое изменение в нем может случайно сломать множество производных классов [50](#).

I — Принцип разделения интерфейсов (Interface Segregation Principle, ISP). Этот

принцип утверждает, что клиенты не должны быть вынуждены зависеть от методов, которые они не используют. Вместо того чтобы иметь один большой и универсальный интерфейс, лучше иметь несколько маленьких и специализированных. Это предотвращает "загрязнение" интерфейсов методами, которые не имеют отношения к конкретному клиенту. Например, если у нас есть интерфейс `IMachine` с методами `Print()`, `Scan()` и `Fax()`, то класс `Printer`, который умеет только печатать, был бы вынужден реализовывать все три метода, даже если они для него не применимы. Гораздо лучше разделить этот интерфейс на `IPrinter`, `IScanner` и `IFax`, и позволить классу `Printer` реализовывать только `IPrinter` [44](#).

D — Принцип инверсии зависимостей (Dependency Inversion Principle, DIP). Этот

принцип является самым абстрактным и глубоким из пяти. Он состоит из двух частей: 1) Модули верхнего уровня не должны зависеть от модулей нижнего уровня. Оба должны зависеть от абстракций. 2) Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций [27](#) [28](#) [68](#). Модулем верхнего уровня является бизнес-логика вашей программы, а модулем нижнего уровня — конкретные технологии, такие как базы данных, сетевые сервисы или UI-фреймворки. DIP говорит о том, что бизнес-логика не должна напрямую использовать классы из фреймворка базы данных. Вместо этого она должна зависеть от абстракций (интерфейсов), определяющих необходимые операции

(например, `IRepository` с методами `Save()`, `FindById()`). А конкретная реализация, работающая с MySQL или PostgreSQL, зависит от этого интерфейса. Это позволяет легко заменять детали (например, поменять базу данных) без изменения кода бизнес-логики, что является ключом к созданию слабосвязанных и гибких систем [45](#) [46](#).

Эти принципы, особенно SOLID, оказывают прямое влияние на фундаментальный выбор в проектировании: наследование или композиция. Хотя наследование является естественным способом переиспользования кода, оно создает очень сильную связь между классами, что может нарушить принципы SRP и LSP. Хрупкость базового класса, когда его изменение ломает всю иерархию, является прямым следствием чрезмерного использования наследования [50](#). Именно поэтому многие опытные архитекторы рекомендуют предпочитать композицию наследованию [30](#) [59](#). Композиция позволяет достигать тех же целей (переиспользование, полиморфизм) с гораздо меньшей связанностью, что делает систему более гибкой, модульной и устойчивой к изменениям. Например, вместо наследования от `List` для создания своего списка, можно создать класс `MyCollection` и сделать его полем-членом объекта класса `List`. Это позволяет использовать все возможности `List`, но не наследует его недостатки и не привязывает себя к его реализации. Таким образом, принципы проектирования служат архитектурным руководством, которое помогает принимать осознанные решения о структуре кода, выбирая между низкоуровневыми механизмами языка (наследование) и высокоуровневыми архитектурными паттернами (композиция, паттерны проектирования).

Порождающие паттерны проектирования

Порождающие паттерны проектирования относятся к категории шаблонов, которые занимаются процессом создания объектов. Их цель — сделать систему независимой от того, как создаются, komponуются и представляются эти объекты. Они достигают этого, инкапсулируя логику создания, что позволяет системе работать с объектами без необходимости знать их конкретные классы. Это особенно важно для создания гибких и расширяемых программ, которые соответствуют принципу открытости/закрытости. Существует несколько ключевых порождающих паттернов, каждый из которых решает свою специфическую задачу.

Фабричный метод (**Factory Method Pattern**) является одним из самых фундаментальных порождающих паттернов. Он определяет интерфейс для создания объекта, но позволяет подклассам решать, какой класс инстанцировать. Фабричный метод откладывает инстанцирование класса в подклассы. Это позволяет семейству классов выбирать, какой класс использовать для создания объекта, не жестко привязываясь к конкретной реализации. Например, у нас есть базовый класс `Document` и его производные `PDFDocument` и `WordDocument`. Вместо того чтобы клиентский код напрямую вызывал конструкторы `new PDFDocument()` или `new WordDocument()`, мы можем определить в классе `Document` виртуальный фабричный метод `virtual unique_ptr<Document> CreateDocument() = 0;`. Затем реализуем его в производных классах: `unique_ptr<Document> PDFDocument::CreateDocument()` вернет `make_unique<PDFDocument>()`. Теперь клиентский код взаимодействует только с базовым классом `Document` и его фабричным методом, не зная ничего о конкретных производных классах. Добавление нового типа документа (например, `EpubDocument`) не требует изменения клиентского кода, только создание нового подкласса и его реализации [13](#) [40](#) [42](#).

Абстрактная фабрика (**Abstract Factory Pattern**) — это более сложный паттерн, который решает проблему создания целых семейств родственных или зависимых продуктов, не специфицируя их конкретные классы. Если фабричный метод создает продукты одного типа, то абстрактная фабрика создает целые "семейства" продуктов, которые должны работать вместе. Например, при разработке кросс-платформенного GUI-приложения мы можем иметь интерфейсы `ITheme`, `IButton`, `ITextField`. Реализации этих интерфейсов будут существовать для разных тем: `WindowsTheme`, `MacTheme` и т.д. `WindowsTheme` будет создавать объекты `WindowsButton` и `WindowsTextField`, которые выглядят и ведут себя как настоящие Windows-компоненты. Клиентский код запрашивает фабрику (`ITheme* theme = new MacTheme();`) и использует ее для создания всех необходимых элементов интерфейса. Это гарантирует, что все созданные элементы будут соответствовать одной и той же теме, и позволяет легко переключать тему приложения без изменения кода, который их использует [16](#) [71](#) [80](#).

Строитель (**Builder Pattern**) предназначен для пошагового построения сложных объектов. Его основное преимущество проявляется, когда конструктору объекта нужно много параметров, некоторые из которых являются необязательными. Вместо того чтобы иметь конструктор с огромным количеством аргументов и

множественных перегрузок, Строитель предлагает использовать объект-строитель, который позволяет определять и настраивать различные части объекта пошагово. Этот паттерн особенно полезен для создания иерархических или вложенных объектов. Например, для создания сложного XML-документа можно иметь класс `XMLBuilder`, который имеет методы `AddElement()`, `SetText()`, `AddAttribute()`. Клиент вызывает эти методы в нужном порядке, а затем вызывает `Build()` для получения готового объекта `XMLDocument`. Это делает код создания объекта более читаемым и понятным по сравнению с длинным конструктором [14 79](#). Строитель не всегда сложнее, чем Фабричный метод, но решает иную проблему: сложность конструкции, а не выбор класса.

Прототип (**Prototype Pattern**) позволяет копировать объекты, не вдаваясь в подробности их реализации. Вместо того чтобы создавать новый объект и затем устанавливать его свойства, мы можем просто скопировать существующий "прототайп" объекта и модифицировать копию. Это особенно эффективно, когда процесс создания объекта очень дорогой по вычислительным затратам. В C++ это реализуется с помощью виртуального метода `clone()`, который в каждом классе-наследнике будет возвращать копию самого себя. Клиентский код взаимодействует только с интерфейсом прототипа, не зная о конкретных классах создаваемых объектов. Это также соответствует принципу открытости/закрытости: чтобы добавить новый тип объекта, достаточно создать новый класс, реализующий интерфейс прототипа, и его копию, не трогая существующий код клиента [80](#).

Синглтон (**Singleton Pattern**) — это простейший из порождающих паттернов, который гарантирует, что у класса есть только один экземпляр, и предоставляет к нему глобальную точку доступа. Этот паттерн часто используется для управления общими ресурсами, такими как доступ к базе данных, файлу логов или конфигурационный файл. Реализация требует сделать конструкторы класса приватными, чтобы никто не мог создать экземпляр вручную. Вместо этого предоставляется публичный статический метод (обычно `getInstance()`), который отвечает за создание единственного экземпляра при первом вызове и возвращает ссылку на него при последующих. Несмотря на свою простоту, Синглтон критикуется за введение глобального состояния, которое может усложнить тестирование (особенно юнит-тестирование) и создать скрытые зависимости между компонентами системы. Тем не менее, при правильном использовании он остается полезным инструментом для решения конкретных задач [70 71](#).

Паттерн	Проблема, которую решает	Когда применять
Фабричный метод	Инкапсуляция процесса создания объекта.	Когда класс не знает, какой класс объекта создавать, или когда класс хочет делегировать создание подклассам.
Абстрактная фабрика	Создание целых семейств родственных продуктов.	Когда система должна быть независимой от способа создания, композиции и представления продуктов.
Строитель	Пошаговое построение сложного объекта.	Когда конструктору нужно много параметров, или когда требуется гибкость в представлении частей сложного объекта.
Прототип	Копирование существующих объектов вместо создания новых через new.	Когда стоимость создания нового объекта высока, или когда клиентский код должен быть независим от классов создаваемых объектов.
Синглтон	Гарантия наличия только одного экземпляра класса.	Для управления общими ресурсами, где требуется глобальная точка доступа (логгеры, кэши).

Структурные и поведенческие паттерны проектирования

После рассмотрения порождающих паттернов, которые сфокусированы на процессе создания объектов, необходимо изучить структурные и поведенческие паттерны. Структурные паттерны описывают, как классы и объекты komponуются для формирования более крупных структур. Они помогают определять отношения между сущностями, такие как наследование, композиция и агрегация, и предлагают более гибкие способы их применения. Поведенческие паттерны, в свою очередь, фокусируются на распределении обязанностей между объектами и на алгоритмах взаимодействия между ними.

Структурные паттерны

Адаптер (Adapter Pattern) — это структурный паттерн, который позволяет объектам с несовместимыми интерфейсами работать вместе. Он действует как "мост" между двумя интерфейсами, преобразуя интерфейс одного класса в интерфейс, ожидаемый клиентом. Это особенно полезно, когда нужно в очередь, логировать или поддерживать отмену (Undo/Redo). Структура включает в себя абстрактный интерфейс **Command** с методами `execute()` и `undo()`, конкретные команды (**MoveCommand**, **DeleteCommand**), которые реализуют этот интерфейс, и **Invoker**, который хранит и выполняет команды. **Invoker** не знает ничего о конкретной реализации команды, он просто вызывает `execute()`. Это позволяет легко реализовать сложные сценарии,

такие как мультикоманда (**MacroCommand**), которая состоит из списка других команд, или история команд для отмены действий [10](#).

- Паттерн Цепочка обязанностей (**Chain of Responsibility**): Этот паттерн позволяет передавать запрос по цепочке обработчиков. Запрос обрабатывается первым подходящим обработчиком в цепочке. Каждый обработчик содержит ссылку на следующий в цепочке (``nextHandler``) и решает, обрабатывать ли запрос сам или передавать его дальше. Это снижает связанность между отправителем запроса и его обработчиками, так как отправитель не знает, кто именно выполнит запрос [11](#).

Сравнение: Оба паттерна инкапсулируют поведение в объекты. Однако **Command** фокусируется на одном запросе и предоставляет ему полный жизненный цикл (выполнение, отмена, повтор), тогда как **Chain of Responsibility** фокусируется на маршрутизации запроса между несколькими потенциальными обработчиками. **Command** — это «что делать», **Chain of Responsibility** — «кому передать».

Паттерны Посетитель и Состояние

- Паттерн Посетитель (**Visitor**): Этот паттерн позволяет определять новую операцию над всеми элементами составной структуры (например, дерева) без изменения классов этих элементов. Он решает проблему «добавить новую операцию к существующей иерархии классов». Структура включает в себя интерфейс ``Visitor``, который объявляет методы ``visit(ConcreteElementA&)``, ``visit(ConcreteElementB&)``, и интерфейс ``Element``, который объявляет метод ``accept(Visitor&)``. Каждый ``ConcreteElement`` реализует ``accept()`` и вызывает соответствующий метод ``visit()`` на переданном посетителе. Это позволяет легко добавлять новые операции (например, ``XMLExportVisitor``, ``JSONExportVisitor``) без изменения кода элементов [11](#).
- Паттерн Состояние (**State**): Этот паттерн позволяет объекту изменять свое поведение, когда его внутреннее состояние изменяется. Он предлагает альтернативу длинным блокам ``if-else`` или ``switch`` по состоянию. Структура включает в себя интерфейс ``State``, который объявляет методы, характерные для состояния, и конкретные классы состояний (``IdleState``, ``RunningState``, ``PausedState``). Контекстный класс (``Context``) содержит указатель на текущее состояние и делегирует вызовы методов ему. При

изменении состояния контекст просто меняет указатель на другой объект состояния [11](#).

Сравнение: Оба паттерна позволяют добавлять новое поведение без изменения существующих классов. Однако **Visitor** добавляет поведение по отношению к структуре данных, тогда как **State** добавляет поведение по отношению к одному объекту. **Visitor** — это «что делать с элементом», **State** — «как вести себя в данном состоянии».

Структурные паттерны проектирования: Организация объектов и классов

Структурные паттерны проектирования описывают, как классы и объекты могут быть составлены для формирования более крупных структур. Они сосредоточены на отношениях между сущностями и на том, как эти отношения могут быть организованы для достижения определенных архитектурных целей, таких как гибкость, простота использования или контроль доступа.

Паттерны Адаптер, Фасад и Заместитель

- Паттерн Адаптер (**Adapter**): Этот паттерн действует как «мост» между двумя несовместимыми интерфейсами. Он позволяет классам работать вместе, не изменяя их исходный код. Структура включает в себя целевой интерфейс (`Target``), адаптируемый класс (`Adaptee``), и адаптер (`Adapter``), который наследует от `Target`` и содержит ссылку на `Adaptee``. Адаптер преобразует вызовы к `Target`` в соответствующие вызовы к `Adaptee``. Это идеальный паттерн для интеграции сторонних библиотек или устаревшего кода [7](#) [43](#).
- Паттерн Фасад (**Facade**): Этот паттерн предоставляет унифицированный, упрощенный интерфейс к сложной подсистеме. Он инкапсулирует сложность, предоставляя клиенту высокий уровень абстракции. Например, `sf::RenderWindow`` сам по себе является фасадом над сложной

подсистемой OpenGL/DirectX. Фасад не добавляет новой функциональности, а лишь упрощает доступ к существующей ⁸.

- Паттерн Заместитель (**Proxy**): Этот паттерн предоставляет объекту-заменитель, который контролирует доступ к другому объекту. Он может выполнять дополнительные действия до или после передачи вызова оригинальному объекту. Существует несколько видов прокси:
- Виртуальный прокси: Создает «тяжелый» объект по требованию (lazy initialization).
- Удаленный прокси: Представляет объект, расположенный в другом адресном пространстве (например, на удаленном сервере).
- Защищающий прокси: Проверяет права доступа перед передачей вызова.
- Кэширующий прокси: Кэширует результаты дорогостоящих операций.
- Логирующий прокси: Логирует все вызовы.

Сравнение: Все три паттерна предоставляют «обертку» вокруг другого объекта, но с разными целями. **Adapter** решает проблему несовместимости интерфейсов, **Facade** решает проблему сложности интерфейса, а **Proxy** решает проблему контроля доступа ⁸.

Паттерны Декоратор, Компоновщик и Мост

- Паттерн Декоратор (**Decorator**): Этот паттерн позволяет динамически добавлять объекту новую функциональность без изменения его структуры. Он предлагает гибкую альтернативу наследованию для расширения поведения. Структура включает в себя компонент (``Component``), конкретный компонент (``ConcreteComponent``) и декоратор (``Decorator``), который наследует от ``Component`` и содержит ссылку на ``Component``. Конкретные декораторы (``ConcreteDecoratorA``, ``ConcreteDecoratorB``) добавляют свою функциональность в методы ``operation()``. В отличие от ``Adapter``, ``Decorator`` сохраняет тот же интерфейс, что и обернутый объект, но расширяет его поведение ⁸.
- Паттерн Компоновщик (**Composite**): Этот паттерн позволяет составлять объекты в древовидные структуры для представления иерархий «часть-целое». Он позволяет клиентскому коду единообразно обрабатывать как отдельные объекты (листья), так и их группы (композиции). Структура включает в себя компонент (``Component``) с методами ``add()``, ``remove()``, ``getChild()``, конкретный компонент (``Leaf``) и композит (``Composite``),

который реализует ``Component`` и управляет коллекцией дочерних компонентов [6](#).

- Паттерн Мост (**Bridge**): Этот паттерн отделяет абстракцию от ее реализации, позволяя им изменяться независимо. Он решает проблему, когда изменения в одной иерархии классов требуют изменений в другой. Структура включает в себя абстракцию (``Abstraction``) и реализацию (``Implementor``). ``Abstraction`` содержит указатель на ``Implementor`` и делегирует часть своей работы ему. Это позволяет создавать две независимые иерархии: одну для абстракций и одну для реализаций [6](#).

Сравнение: **Decorator** и **Composite** оба используют рекурсивную структуру, но с разными целями. **Decorator** добавляет поведение, тогда как **Composite** управляет структурой. **Bridge** и **Strategy** оба разделяют абстракцию и реализацию, но **Bridge** делает это для архитектурной декомпозиции, тогда как **Strategy** — для выбора алгоритма.

Порождающие паттерны проектирования: Контроль над созданием объектов

Порождающие паттерны проектирования сосредоточены на механизмах создания объектов. Они предоставляют гибкие и контролируемые способы инстанцирования объектов, что позволяет избежать жесткой привязки клиентского кода к конкретным классам и упрощает управление жизненным циклом объектов.

Паттерны Фабричный метод, Абстрактная фабрика и Строитель

- Паттерн Фабричный метод (**Factory Method**): Этот паттерн определяет интерфейс для создания объекта, но оставляет подклассам решение о том, какой класс инстанцировать. Он делегирует создание объектов подклассам. Структура включает в себя абстрактный класс ``Creator`` с виртуальным методом ``factoryMethod()``, который возвращает ``Product``, и конкретные классы ``ConcreteCreatorA``, ``ConcreteCreatorB``, которые переопределяют этот метод для создания различных ``ConcreteProductA``,

``ConcreteProductB``. Это позволяет семейству классов выбирать, какой конкретный продукт создавать, не жестко привязываясь к его имени [13](#) [40](#).

- Паттерн Абстрактная фабрика (**Abstract Factory**): Этот паттерн предоставляет интерфейс для создания семейств связанных или зависимых объектов без указания их конкретных классов. Он является расширением паттерна ``Factory Method``. Структура включает в себя абстрактную фабрику (``AbstractFactory``) с методами ``createProductA()``, ``createProductB()``, и конкретные фабрики (``ConcreteFactory1``, ``ConcreteFactory2``), которые создают продукты одной вариации (например, ``WindowsButton`` и ``WindowsCheckbox`` или ``MacButton`` и ``MacCheckbox``) [16](#) [80](#).
- Паттерн Строитель (**Builder**): Этот паттерн разделяет построение сложного объекта и его представление, позволяя использовать один и тот же процесс построения для создания различных представлений. Он решает проблему создания объектов с большим количеством параметров, некоторые из которых могут быть опциональными. Структура включает в себя интерфейс ``Builder``, конкретных строителей (``ConcreteBuilderA``, ``ConcreteBuilderB``), директора (``Director``), который управляет процессом построения, и продукт (``Product``). ``Director`` вызывает методы ``Builder`` в определенном порядке, а ``Builder`` собирает объект пошагово [14](#) [79](#).

Сравнение: **Factory Method** и **Abstract Factory** решают проблему создания одного объекта или семейства объектов соответственно. **Builder** решает проблему создания одного сложного объекта с множеством настроек. **Factory Method** — это класс-создатель, **Abstract Factory** — это фабрика фабрик, **Builder** — это мастер-строитель.

Паттерны Прототип и Синглтон

- Паттерн Прототип (**Prototype**): Этот паттерн позволяет создавать новые объекты путем копирования существующего объекта (прототипа) вместо создания их через конструкторы. Он полезен, когда создание объекта дорогостояще или когда необходимо создать объект, конфигурация которого уже известна. Структура включает в себя интерфейс ``Prototype`` с

методом ``clone()``, и конкретные классы, которые реализуют этот интерфейс и возвращают копию себя [70](#) [71](#).

- Паттерн Синглтон (**Singleton**): Этот паттерн гарантирует, что у класса есть только один экземпляр, и предоставляет к нему глобальную точку доступа. Он используется для управления общими ресурсами, такими как логгер, конфигурационный менеджер или менеджер подключений к базе данных. Структура включает в себя приватный конструктор, статическое поле-указатель на экземпляр и статический метод ``getInstance()``, который возвращает этот экземпляр. Современная реализация часто использует ``std::call_once`` и ``std::once_flag`` для потокобезопасной инициализации [70](#) [71](#).

Сравнение: Оба паттерна контролируют количество создаваемых экземпляров. **Prototype** контролирует как создавать объекты (через копирование), тогда как **Singleton** контролирует сколько объектов может существовать (ровно один).

Заключение: Синтез знаний для успешной подготовки

Подготовка к коллоквиуму по C++ — это не сбор разрозненных фактов, а построение целостной, многоуровневой архитектуры знаний. Как было продемонстрировано в этом конспекте, все темы, от конкретных API библиотеки SFML до абстрактных принципов проектирования, образуют единую иерархическую структуру, где каждый уровень логически вытекает из предыдущего и служит основой для последующего.

Событийно-ориентированное программирование в SFML — это не просто набор функций для рисования и обработки мыши. Это практическая реализация архитектурного стиля, который требует от разработчика понимания того, как управлять состоянием приложения, как обрабатывать асинхронные события и как организовать код для обеспечения отзывчивости. Именно здесь начинается применение всех остальных концепций: для создания интерактивных элементов (**Draggable**, **Button**) используется наследование; для управления их поведением — динамический полиморфизм; для организации их взаимодействия — поведенческие паттерны, такие как **Observer**.

Наследование и динамический полиморфизм — это не просто синтаксические конструкции языка, а мощные архитектурные инструменты. Понимание тонкостей наследования (затенение vs. переопределение, виртуальное наследование, порядок вызовов конструкторов) — это ключ к написанию кода, который не будет ломаться при расширении. Понимание механизма vtable и RTTI — это ключ к написанию кода, который будет безопасно и эффективно работать с иерархией классов во время выполнения.

Принципы проектирования и паттерны проектирования — это не «надстройка» над языком, а его логическое завершение. Они являются результатом многолетнего опыта и представляют собой проверенные решения для типовых проблем. Принципы SOLID дают ответ на вопрос «почему» стоит поступать так, а не иначе. Паттерны дают ответ на вопрос «как» это сделать. Паттерн **Strategy** — это практическая реализация принципа «инкапсулируйте то, что может измениться». Паттерн **Builder** — это практическая реализация принципа единственной ответственности. Паттерн **Adapter** — это практическая реализация принципа разделения интерфейсов.

Таким образом, успешная подготовка к коллоквиуму требует не заучивания, а синтеза. Необходимо уметь видеть связь между `sf::Event::KeyPressed` и паттерном **Observer**, между `virtual` деструктором и принципом подстановки Барбары Лисков, между `dynamic_cast` и безопасностью работы с иерархией наследования. Только такое глубокое, системное понимание позволит не просто ответить на вопросы коллоквиума, а продемонстрировать истинное владение предметом — искусством создания надежных, гибких и поддерживаемых программных систем на языке C++.

Справка

1. Static cast vs. dynamic cast for traversing inheritance hierarchies <https://stackoverflow.com/questions/1382540/static-cast-vs-dynamic-cast-for-traversing-inheritance-hierarchies>
2. Order of member constructor and destructor calls - Stack Overflow <https://stackoverflow.com/questions/2254263/order-of-member-constructor-and-destructor-calls>

3. Inheritance in C++ (Syntax, All Types With Examples) - WsCube Tech <https://www.wscubetech.com/resources/cpp/inheritance>
4. c++ - How does virtual inheritance solve the "diamond" (multiple ... <https://stackoverflow.com/questions/2659116/how-does-virtual-inheritance-solve-the-diamond-multiple-inheritance-ambiguit>
5. Diamond Problem in C++ - GeeksforGeeks <https://www.geeksforgeeks.org/cpp/diamond-problem-in-cpp/>
6. Gang of 4 Design Patterns Explained: Creational, Structural, and ... <https://www.digitalocean.com/community/tutorials/gangs-of-four-gof-design-patterns>
7. Adapter Design Pattern - GeeksforGeeks <https://www.geeksforgeeks.org/system-design/adapter-pattern/>
8. Difference between the Facade, Proxy, Adapter and Decorator ... <https://stackoverflow.com/questions/3489131/difference-between-the-facade-proxy-adapter-and-decorator-design-patterns>
9. Proxy Design Pattern in Modern C++ - DEV Community <https://dev.to/visheshpatel/proxy-design-pattern-in-modern-c-aj9>
10. Behavioural Design Patterns in C++ - TutorialsPoint https://www.tutorialspoint.com/cplusplus/cpp_behavioural_design_patterns.htm
11. Behavioral Design Patterns - GeeksforGeeks <https://www.geeksforgeeks.org/system-design/behavioral-design-patterns/>
12. Design Patterns in C++ - Behavioral - Observer to Visitor_哔哩哔哩 <https://www.bilibili.com/video/BV1gC4y1W79M/>
13. How to implement the factory method pattern in C++ correctly <https://stackoverflow.com/questions/5120768/how-to-implement-the-factory-method-pattern-in-c-correctly>
14. Builder Design Pattern in Modern C++ - DEV Community <https://dev.to/visheshpatel/builder-design-pattern-in-modern-c-2ipd>
15. Design Patterns: Exploring Factory Method in Modern C++ <https://hackernoon.com/desing-patterns-exploring-factory-method-in-modern-c-hi1h3uvw>
16. Abstract Factory Design Pattern in C++ - TutorialsPoint https://www.tutorialspoint.com/cplusplus/cpp_abstract_factory_design_pattern.htm
17. Why do we need a pure virtual destructor in C++? - Stack Overflow <https://stackoverflow.com/questions/1219607/why-do-we-need-a-pure-virtual-destructor-in-c>
18. Vtable placement of completely pure-virtual class - Stack Overflow <https://stackoverflow.com/a/4618389>
19. Pure Virtual Destructor in C++ - GeeksforGeeks <https://www.geeksforgeeks.org/cpp/pure-virtual-destructor-c/>

20. C++ 对象模型：RTTI 的实现原理与类型识别机制原创 - CSDN博客 https://blog.csdn.net/qq_40529346/article/details/150600055
21. C++ 中的RTTI机制、typeid和dynamic_cast、多继承中的虚函数 - 博客园 <https://www.cnblogs.com/codingbigdog/p/17780262.html>
22. 【C++ 多态】深入理解C++的运行时类型信息（RTTI） <https://developer.aliyun.com/article/1466339>
23. C++ RTTI 机制下判断类型的typeid 和记录类型信息的type_info 原创 ... https://blog.csdn.net/m0_37701628/article/details/117490063
24. C++：从技术实现角度聊聊RTTI - 腾讯云 <https://cloud.tencent.com/developer/article/2295955>
25. C++ 中dynamic_cast与RTTI的依赖关系及编译控制_文心快码 - comate <https://comate.baidu.com/zh/page/52407qzh4yj>
26. C++ 多重继承和虚拟继承的内存布局（vtable,vptr） 转载 - CSDN博客 <https://blog.csdn.net/u013617648/article/details/46582367>
27. SOLID Principles with Real Life Examples - GeeksforGeeks <https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/>
28. SOLID Design: Dependency Inversion Principle in C++ | HackerNoon <https://hackernoon.com/solid-design-dependency-inversion-principle-in-c-weq3ujp>
29. Advantages/Disadvantages of a Inheritance, composition and ... <https://stackoverflow.com/questions/6246106/advantages-disadvantages-of-a-inheritance-composition-and-multiple-member-varia>
30. Composition Over Inheritance: A Flexible Design Principle <https://dev.to/lovestaco/composition-over-inheritance-a-flexible-design-principle-4ehh>
31. Composition vs. Inheritance: How to Choose? - Thoughtworks <https://www.thoughtworks.com/insights/blog/composition-vs-inheritance-how-choose>
32. Why composition is favored over inheritance - LinkedIn <https://www.linkedin.com/pulse/why-composition-favored-over-inheritance-shehan-chanuka>
33. [PDF] CZ: Multiple Inheritance Without Diamonds <https://www.cs.cmu.edu/~aldrich/FOOL09/malayeri.pdf>
34. Order of Constructor/ Destructor Call in C++ - GeeksforGeeks <https://www.geeksforgeeks.org/cpp/order-constructor-destructor-call-c/>
35. C++ Constructor/Destructor inheritance - Stack Overflow <https://stackoverflow.com/questions/14184341/c-constructor-destructor-inheritance>
36. Inheritance and Access Specifiers | PDF | Namespace - Scribd <https://www.scribd.com/presentation/855313660/c-1>

37. static cast versus dynamic cast - c++ - Stack Overflow <https://stackoverflow.com/questions/1254777/static-cast-versus-dynamic-cast/1254793>
38. C++ 编译器检索VTABLE的具体方法不同原创 - CSDN博客 <https://blog.csdn.net/techweb/article/details/2265485>
39. 【现代C++特性】RTTI运行阶段类型识别 (Runtime Type ... - 稀土掘金 <https://juejin.cn/post/7470366770835505186>
40. 【C++ Design Pattern-Creational】Factory Method (工厂方法) 原创 https://blog.csdn.net/KG_Qt/article/details/7232413
41. Creational Pattern in C++ - GeeksforGeeks <https://www.geeksforgeeks.org/cpp/creational-pattern-in-c/>
42. Creational Design Patterns in Modern C++ - Coursera <https://www.coursera.org/learn/packt-creational-design-patterns-in-modern-c-5n9at>
43. Structural Design Patterns - GeeksforGeeks <https://www.geeksforgeeks.org/system-design/structural-design-patterns/>
44. SOLID principles implementation for C - Stack Overflow <https://stackoverflow.com/questions/17648266/solid-principles-implementation-for-c>
45. Mastering C# Programming: Best Skills, SOLID Principles, and Key ... <https://www.linkedin.com/pulse/mastering-c-programming-best-skills-solid-principles-key-badugu-hosyc>
46. Implementation of a Universal Framework Using Design Patterns for ... <https://www.mdpi.com/1424-8220/24/10/3116>
47. Software Development and Professional Practice - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-1-4302-3802-7.pdf>
48. Peeling Design Patterns For Beginners Narasimha ... - Scribd <https://www.scribd.com/document/372230370/peeling-design-patterns-for-beginners-narasimha-karumanchi8091-www-ebook-dl-com>
49. Software Development, Design and Coding - pdfcoffee.com <https://pdfcoffee.com/software-development-design-and-coding-pdf-free.html>
50. Composition Over Inheritance - LinkedIn <https://www.linkedin.com/pulse/composition-over-inheritance-tracy-atteberry-1gmoc>
51. Composition V.S Inheritance , Which one to choose & Why? <https://dev.to/smuhammed/composition-vs-inheritance-which-one-to-choose-why-3ndm>
52. C++ Public, Protected and Private Inheritance - Programiz <https://www.programiz.com/cpp-programming/public-protected-private-inheritance>
53. What are access specifiers? Should I inherit with private, protected ... <https://stackoverflow.com/questions/5447498/what-are-access-specifiers-should-i-inherit-with-private-protected-or-public>

54. C++ Inheritance Access - GeeksforGeeks <https://www.geeksforgeeks.org/cpp/cpp-inheritance-access/>
55. Access Specifiers and Inheritance in C++ | PDF - Scribd <https://www.scribd.com/presentation/918523139/Opp-22222>
56. static_cast and RTTI vs dynamic_cast - c++ - Stack Overflow <https://stackoverflow.com/questions/12588264/static-cast-and-rtti-vs-dynamic-cast>
57. C++ Tricks: Fast RTTI and Dynamic Cast - DEV Community <https://dev.to/kahncode/c-tricks-fast-rtti-and-dynamic-cast-3k2f>
58. Understanding the Diamond Problem in C++ | PDF - Scribd <https://www.scribd.com/document/825175760/Diamond-Problem>
59. Prefer composition over inheritance? - Stack Overflow <https://stackoverflow.com/questions/49002/prefer-composition-over-inheritance>
60. Clean Architecture - Lewis, Elijah (Three Books Bundle) | PDF - Scribd <https://www.scribd.com/document/695200559/Clean-Architecture-Lewis-Elijah-Three-Books-Bundle>
61. Adapter Design Pattern in Modern C++ - DEV Community <https://dev.to/visheshpatel/adapter-design-pattern-in-modern-c-h56>
62. C++ Classes and Inheritance Overview | PDF - Scribd <https://www.scribd.com/document/868716509/Unit-4>
63. Multiple Inheritance Virtual Call Ambiguity - c++ - Stack Overflow <https://stackoverflow.com/questions/31521743/multiple-inheritance-virtual-call-ambiguity>
64. Multiple Inheritance & Virtual Inheritance in Visual C++ - jackminnet <https://jackmin.home.blog/2018/12/23/multiple-inheritance-virtual-inheritance-in-visual-c/>
65. Memory Layout for Multiple and Virtual Inheritance - 博客园 <https://www.cnblogs.com/yetanghanCpp/p/9288708.html>
66. Understanding the Diamond Problem in C++ | PDF - Scribd <https://www.scribd.com/document/68590382/Multiple-Inheritance-Diamond-Problem-in-C>
67. 【全网首发】C++：从技术实现角度聊聊RTTI - HeapDump性能社区 <https://heapdump.cn/article/5109918>
68. oop - Dependency inversion (from S.O.L.I.D principles) in C++ <https://stackoverflow.com/questions/43576556/dependency-inversion-from-s-o-l-i-d-principles-in-c>
69. The D in SOLID stands for the "Dependency inversion principle". https://www.linkedin.com/posts/jakob-jenkov-4a3a8_the-d-in-solid-stands-for-the-dependency-activity-7339562759728037889-8wyU
70. Creational Design Patterns in C++ - TutorialsPoint https://www.tutorialspoint.com/cplusplus/cpp_creational_design_patterns.htm

71. Introduction to Creational Design Patterns - Baeldung <https://www.baeldung.com/creational-design-patterns>
72. c++ - When should static_cast, dynamic_cast, const_cast, and ... <https://stackoverflow.com/questions/332030/when-should-static-cast-dynamic-cast-const-cast-and-reinterpret-cast-be-used>
73. Usage of `dynamic\_cast` in Blender code - Technical Feedback <https://devtalk.blender.org/t/usage-of-dynamic-cast-in-blender-code/35609>
74. Diamond inheritance in Multiple Inheritance using the Virtual keyword <https://stackoverflow.com/questions/29782593/diamond-inheritance-in-multiple-inheritance-using-the-virtual-keyword>
75. Memory Layout of Virtual Inherit (虚继承的内存布局) 原创 - CSDN博客 <https://blog.csdn.net/fishhg/article/details/6283504>
76. Inheritance in C++ - GeeksforGeeks <https://www.geeksforgeeks.org/cpp/inheritance-in-c/>
77. C++ Inheritance and Constructors Guide - Programming - Scribd <https://www.scribd.com/presentation/889980574/Module-3-Inheritance>
78. 深入理解C++对象模型--vtable 布局 - 知乎专栏 <https://zhuanlan.zhihu.com/p/496115833>
79. What is the difference between Builder Design pattern and Factory ... <https://stackoverflow.com/questions/757743/what-is-the-difference-between-builder-design-pattern-and-factory-design-pattern>
80. Design-Patterns (C++ Examples) | Class (Computer Programming) <https://www.scribd.com/document/459721898/Design-Patterns-C-examples>